

Delayed Actions for Unity

Delayed Actions	2
Features	2
How It Works	2
Basic Usage	4
Scheduling Actions	4
Executors	5
Locators	5
Cancelling Actions	6
Querying Actions	8
Sequences	9
Groups	11
Debug Mode	12
Notes and Limitations	13
Best Practices	13
API Cheatsheet	14
Do	14
Abort	14
Sequences	14
Queries	15
Groups	15

Delayed Actions

Lightweight Delayed Execution Without Coroutines

Delayed Actions is a lightweight Unity utility that allows you to schedule the execution of C# *Action* delegates after a delay without using Coroutines, *Invoke()*, or manually managing timers in *Update()*.

The library is designed to be simple, efficient, and easy to integrate into existing projects while still providing useful features such as cancellation, action querying, action grouping, and sequence building.

This library was created by Miguel Lázaro Gil in 2026. lazaro.gil.miguel@gmail.com

Features

- Schedule actions to execute after a delay.
 - No Coroutines or *Invoke()* required.
 - Uses Unity game time (*Time.deltaTime*).
 - Automatically pauses timers when the executor is disabled.
 - Automatically cancels actions when the executor is destroyed.
 - Cancel actions individually, by locator, by executor, or by group.
 - Query pending actions.
 - Create action sequences with waits and loops.
 - Supports multiple independent action groups.
 - Built-in editor debugging (optional).
-

How It Works

Delayed Actions creates an internal *GameObject* the first time it is used.

```
DontDestroyOnLoad
├── DelayedActions
│   ├── _DelayedActions_(DA)
│   ├── _EnemyAI_(DA)
│   └── _Audio_(DA)
```

Each group manages its own scheduled actions.

Every frame:

- Delays are decreased using *Time.deltaTime*.
- Expired *actions* are executed.
- Cancelled *actions* are removed.
- Destroyed *executors* automatically lose all pending actions.

Because timers are updated during *Update()*, execution accuracy is approximately **±1 frame**.

Basic Usage

The simplest way to schedule an action is:

```
DelayedActions.Do(
    this,
    2f,
    () => Debug.Log("Hello!")
);
```

After two seconds the message will be printed.

Scheduling Actions

There are four overloads available:

- *DelayedActions.Do(executor, delay, action);*
- *DelayedActions.Do(executor, delay, action, locator);*
- *DelayedActions.Do(executor, delay, action, actionName);*
- *DelayedActions.Do(executor, delay, action, locator, actionName);*

Parameter	Description
executor	<i>MonoBehaviour</i> responsible for the action.
delay	Delay in seconds.
action	<i>C# Action</i> delegate.
locator	Optional identifier used to cancel multiple actions together.
actionName	Optional debug name shown in the inspector.

Returns	Description
id	Unique long id to identify each action.

Executors

Every action belongs to a *MonoBehaviour* **executor**.

Executors are mandatory because they define the lifetime of scheduled actions.

If the *GameObject* is destroyed:

- all pending actions are automatically cancelled.

If the component is disabled:

- timers stop advancing until it becomes active again.

Most of the time the executor will be *this*. Keep in mind the executor should be the one who runs the code.

Locators

A *locator* is an optional integer assigned by the user.

Unlike IDs, locators are **not unique**.

Several actions may share the same *locator*.

Example:

- *DelayedActions.Do(this, 1, Jump, 5);*
- *DelayedActions.Do(this, 2, Run, 5);*
- *DelayedActions.Do(this, 3, Attack, 5);*

All three actions can later be cancelled together:

- *DelayedActions.Abort(this, 5);*

This makes locators useful for cancelling **logical groups** of actions.

Cancelling Actions

Cancel everything belonging to one executor

DelayedActions.Abort(this);

Every action in the group scheduled under said executor will be cancelled.

Cancel all actions with a locator

DelayedActions.Abort(10);

Every action in the group with said locator will be cancelled.

Cancel locator under one executor

DelayedActions.Abort(this, 10);

Every action with said locator under said executor will be cancelled (more efficient than the previous version if only looking at one executor).

Cancel by ID

DelayedActions.AbortById(id);

Only the action matching that unique ID is cancelled.

Cancel an entire group

DelayedActions.AbortGroup();

Every scheduled action inside the current DelayedActions group is cancelled.

Querying Actions

The library lets you inspect scheduled actions. Queries of action are group dependent, to query an action scheduled in a specific group you must query using said group.

Check if still pending

```
bool pending = DelayedActions.IsScheduled(id);
```

If the action has been executed or has never been scheduled returns false.

Time remaining

```
double remaining = DelayedActions.GetTimeLeft(id);
```

Returns a negative value if the action no longer exists.

Access the action object

```
DelayedActionInfo info = DelayedActions.GetAction(id);
```

This gives direct access to:

- action
- remaining delay
- locator
- cancellation state
- debug name **

Changing these values directly affects the scheduled action. You cannot change its group or executor.

**Action's name is only compiled with *UnityEditor*, do not access that field unless using *UnityEditor* (calls like *Do* simply ignore the name, those will compile but this one will not)

Sequences

Sequences provide a fluent interface for scheduling multiple actions.

Example:

- *DelayedActions.Sequence(this)*
.Wait(2)
.Do(SpawnBoss,"Boss")
.Wait(3)
.Do(OpenDoor,"Door")
.Start();

Timeline:

2 sec -> Spawn Boss -> 3 sec later -> Open Door

Functions:

- *Sequence(executor)*
- *Wait(seconds)*
- *Do(action , actionName)*
- *Section()*
- *Loop(timesPlayed , delayBetweenIter)*
- *Start(locator) **
- *LoopInfinite(locator) **

Running a sequence returns the assigned id of the list of actions.

- *List<long> ids;*

*These functions will cause the actions to be scheduled.

Sections and Loops

Loops allow for repetition of *sections*. There is an implicit section start when creating a *sequence*.

- *DelayedActions.Sequence(this)*
.Wait(4)
.Do(TakeCover)
.Section()
.Do(Shoot)
.Wait(1)

```
.Loop(3)
.Start();
```

Timeline:

4 sec -> TakeCover -> Shoot -> 1 sec -> Shoot -> 1 sec -> Shoot -> 1 sec

Loops support nesting.

```
.Section()
...
.Section()
...
.Loop(3)
.Loop(2)
```

Nested *sections* are consumed by the nearest *loop*.

Loops are **unrolled**, you may see multiple actions being scheduled.

Infinite Loops

The last *section* repeats forever.

- *DelayedActions.Sequence(this)*

```
.Wait(4)
.Do(TakeCover)
.Section()
.Do(Shoot)
.Wait(0.5)
.LoopInfinite();
```

4 sec -> TakeCover -> Shoot -> 0.5 sec -> Shoot -> 0.5 sec -> Shoot ...

Only the actions inside the last section are repeated.

Everything before the section executes once.

This function consumes the *Sequence Builder*, it must not be reused afterwards.

The *ids* returned by this function correspond to the non repeated code and the first iteration of the loop. Further actions will not be accessible by *id*.

Groups

By default the library uses genericity to create multiple static classes on the go:

DelayedActions internally this is simply *DelayedActions<DelayedActions>*

- By default all actions will be scheduled inside the DelayedActions group.

Create groups as pleased using genericity:

- DelayedActions<EnemyAI>.Do(...);
- EnemyAI would be a C# class and the action will be scheduled in the EnemyAI group.
- The gameObject containing the actions will be called “_EnemyAI_(DA)”

There is virtually no constraint in which classes can be used to create groups.

You can create “shortcuts”:

- public class DelayedSounds : DelayedActions<DelayedSounds>{}
- DelayedSounds.Sequence(...)
- DelayedSounds.AbortGroup();

Each group has:

- its own scheduled actions
- its own GameObject
- its own AbortGroup
- Aborting functions of a group will never affect other groups; ie aborting by locator in one group will not abort another group’s action even if the locator is the same.

IDs remain globally unique across all groups, but anything regarding *IDs* **must be called on the same group that scheduled the action.**

Debug Mode

When compiled in the *Unity Editor* the library includes an inspector.

It displays:

- executors
- scheduled actions
- remaining time
- locators
- IDs
- action names

Actions can also be:

- cancelled
- renamed
- edited
- inspected in real time

To disable debug mode, comment out:

```
#define DebugModeActive
```

When disabled:

- no debug names are stored
- no inspector is generated
- no debug logs are produced

This matches build behaviour and has better performance.

Notes and Limitations

- Uses Unity game time (`Time.deltaTime`).
 - Affected by `Time.timeScale`.
 - Accuracy is approximately one frame.
 - Executors cannot be `null`.
 - Actions cannot be `null`.
 - Disabled components pause timers.
 - Destroyed executors automatically cancel pending actions.
 - Individual pause/resume of actions is not supported.
 - The library schedules only `Action` delegates.
-

Best Practices

- Always provide meaningful action names while developing.
 - Use locators to group related actions.
 - Use `Abort(executor, locator)` whenever possible for better performance.
 - Use IDs only when you need to track a specific action.
 - Use groups to separate unrelated systems (AI, Audio, UI, etc.).
 - Use sequences instead of deeply nested delayed callbacks for better readability.
-

API Cheatsheet

	Description
executor	<i>MonoBehaviour</i> responsible for the action.
delay	Delay in seconds.
action	<i>C# Action</i> delegate.
locator	Optional identifier used to cancel multiple actions together.
actionName	Optional debug name shown in the inspector.

Do

- `public long Do(MonoBehaviour executor, double delay, Action action);`
- `public long Do(MonoBehaviour executor, double delay, Action action, int locator);`
- `public long Do(MonoBehaviour executor, double delay, Action action, string actionName);`
- `public long Do(MonoBehaviour executor, double delay, Action action, int locator, string actionName);`

Abort

- `public void Abort(MonoBehaviour executor);`
- `public void Abort(MonoBehaviour executor, int locator);`
- `public void Abort(int locator);`
- `public void AbortById(long id);`
- `public void AbortGroup();`

Sequences

- `public SequenceBuilder Sequence(MonoBehaviour executor);`
- `public Wait(double seconds);`
- `public Do(Action action, string actionName);`
- `public Section();`
- `public Loop(int totalTimesExecuted=2, double iterationDelay=0);`
- `public List<long> Start(int locator=0);`

- `public List<long> LoopInfinite(double iterationDelay = 0, int locator = 0);`

Queries

- `public bool IsScheduled(long id);`
- `public double GetTimeLeft(long id);`
- `public DelayedActionInfo GetAction(long id);`

DelayedActionInfo contains:

- Action
- Delay
- Cancelled
- Name
- Locator
- Id (readonly)

Groups

- DelayedActions is the default group
- `DelayedActions<MyClass>.Do(...)` automatically creates a new group and uses it
- Shortcuts:
 - `class DelayedSounds : DelayedActions<DelayedSounds>{}`